



UPA - projekt 2

Team - xkotou08

Lukáš Kotoun - xkotou08

Jan Pánek - xpanek11

Jiří Kantor - xkanto16

26. října 2025

Sloupcová databáze - Apache Cassandra

Datová sada a distribuce

- Název: Zatížení přepážek
- Url: <https://opendata.plzen.eu/public/opendata/detail/165>
- Distribuce: CSV

Vysvětlení volby

Jedná se o data vyvolávacích systémů přepážkových pracovišť z Plzeňského úřadu ve formátu CSV. Každý záznam obsahuje až 16 položek (*klientID*, *typZaznamu*, *sluzba*, *cisloListku*, *pre-pazka*, *rezervace*, *casRezervace*, *propadRezervace*, *prichod*, *volani*, *opakovaneVolaniMin*, *opakovaneVolaniMax*, *opakovaneVolaniPocet*, *odchod*, *odchodPoPreposlani*, *odchodNeprisel*). Z nich jsou vždy vyplněny pouze položky *klientID*, *typZaznamu*, *sluzba* a *rezervace*, zatímco ostatní se mohou vyskytovat nepravidelně. Většina záznamů obsahuje údaje o příchodu a odchodu klienta (*prichod*, *odchod*), což platí pro záznamy s hodnotou *typZaznamu* = 0. Pokud má tato položka hodnotu 4, je naopak vždy vyplněna položka *propadRezervace*. Hodnota *klientID* je unikátní napříč celou datovou sadou. Soubor aktuálně obsahuje více než 1,2 milionu záznamů, přičemž nové záznamy jsou doplňovány pravidelně každý týden již od roku 2021 a nepředpokládá se budoucí upravování či přepis záznamu.

Pro ukládání a dotazování nad touto datovou sadou byla zvolena sloupcově orientovaná NoSQL databáze Apache Cassandra, jejíž datový model je vhodný pro rozsáhlé, nepravidelně strukturované a průběžně rostoucí záznamy. Cassandra umožňuje efektivní uložení dat, která mají část sloupců s častým výskytem a část pouze občas vyplněnou, což odpovídá právě této sadě, kde jsou některé položky často prázdné. Cassandra je navržena pro velké objemy dat s vysokou četností přidávání nových záznamů, což přesně odpovídá tomuto datasetu, kde se data průběžně doplňují a zpětně nemění. Výhodou je také lineární škálovatelnost a distribuované ukládání, které umožňuje horizontální rozšiřování systému při růstu objemu dat bez nutnosti měnit strukturu nebo přenastavovat aplikaci.

Volba primárního klíče ve tvaru PRIMARY KEY ((*sluzba*), *klientID*) umožňuje efektivní seskupování záznamů podle poskytované služby, což je nejlogičtější dělení vzhledem k charakteru dat (neexistuje vhodný časový ani jiný údaj pro clustering). Tento návrh umožňuje rychlé vyhledávání a agregaci dat podle služby (například pro analýzy vytíženosti jednotlivých služeb) a zároveň zachovává jednoznačnost jednotlivých záznamů díky *klientID*.

Příkaz pro definici

Napojení na databázi Apache Cassandra v jazyce python má následující podobu:

```
cluster = Cluster([cassandradb])
session = cluster.connect()
```

Tento kód vytvoří spojení na adresu `cassandradb` a výchozí port 9042. Poté je potřeba vytvořit keyspace a následně ho vybrat příkazem USE. V jazyce CQL je to provedeno následujícím

příkazem (v souboru **cassandraDB.py** toto zajišťuje funkce **create_keyspace**), který vytvoří keyspace s názvem **cassandratest** a zároveň ho vybere:

```
CREATE KEYSPACE IF NOT EXISTS cassandratest WITH \
replication = {'class': 'SimpleStrategy', 'replication_factor' : 1};
USE cassandratest;
```

Dále je potřeba definovat strukturu pro data, která budeme do databáze ukládat. V jazyce CQL to vypadá následovně:

```
CREATE TABLE IF NOT EXISTS zaznamy (
    klientID int,
    casRezervace timestamp,
    cisloListku int,
    typZaznamu int,
    sluzba int,
    prepazka int,
    rezervace text,
    propadRezervace timestamp,
    prichod timestamp,
    volani timestamp,
    opakovaneVolaniMin timestamp,
    opakovaneVolaniMax timestamp,
    opakovaneVolaniPocet int,
    odchod timestamp,
    odchodPoPreposlani timestamp,
    odchodNeprisel timestamp,
    PRIMARY KEY ((sluzba), klientID)
);
```

V přiloženém python skriptu **cassandraDB.py** se o to stará funkce **create_table**, která pouze spustí tento definiční příkaz

Algoritmus na import dat

Algoritmus na import dat lze vidět ve funkci **load_csv_data** v přiloženém souboru **cassandraDB.py**. Jedná se o předpřipravení databázového výrazu pro vložení jednotlivých záznamů, který je posléze prováděn po dávkách. Výraz má následující podobu:

```
INSERT INTO zaznamy (
    klientID, typZaznamu, sluzba, cisloListku, prepazka,
    rezervace, casRezervace, propadRezervace, prichod, volani,
    opakovaneVolaniMin, opakovaneVolaniMax, opakovaneVolaniPocet,
    odchod, odchodPoPreposlani, odchodNeprisel
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

Dotaz nad daty

Dotaz nad daty lze vidět ve funkci **query_sample** a **query_sample1** v příloženém souboru **cassandraDB.py**. Jako partition key bylo zvoleno číslo služby (*sluzba*) a jako clustering key číslo klienta (*klientID*). Data jsou tedy v databázi rozdělena podle služby do jednotlivých částí (partitions). Uvnitř každé služby jsou pak záznamy pro jednotlivé klienty uspořádány podle klientského čísla.

První dotaz vrací všechny záznamy klientů pro konkrétní službu. Využívá partition key *sluzba*, který určuje, na jakém uzlu se daná data fyzicky nachází, a clustering key *klientID*, který zajišťuje jejich efektivní uložení a seřazení v rámci partition. Pro ukázkou je vypsané pouze 20 řádků.

Při zpracování dotazu Cassandra nejprve pomocí tzv. partitioneru vypočítá hash hodnotu klíče *sluzba* = 4, podle které nalezne uzly, na nichž se partition nachází. Koordinátor následně pošle dotaz přímo těmto uzlům, které data načtou z lokálních SSTable souborů. Výsledky jsou poté agregovány a doručeny klientovi. Tento mechanismus umožňuje distribuované, škálovatelné a velmi rychlé zpracování dotazů bez nutnosti komplexních spojování dat jako v relačních databázích. Pokud bychom však chtěli vybírat záznamy podle jiného sloupce, ztratila by se efektivita a rychlost dotazu a při dotazech by se musel přidat parametr *allow filtering*. To je ilustrativně ukázáno v druhém dotazu.

Dokumentová databáze - MongoDB

Datová sada a distribuce

- Název: Myslivecké honitby
- Url: <https://opendata.plzen.eu/public/opendata/detail/96>
- Distribuce: GeoJSON

Vysvětlení volby

Jedná se o data s atributy a geografickými údaji (polygony a body), které jsou reprezentovány jako vnořené pole. Některé atributy sdílí všechny položky (např. OBJEKT, NAZEV, ID_HON), jiné se vyskytují pouze u některých záznamů (např. text). Dokumentová databáze umožňuje uložit tyto záznamy bez pevné struktury. Sada neobsahuje časové údaje ani vazby mezi jednotlivými honitbami, proto není vhodná pro grafové databáze ani databáze časových řad. Zároveň obsahuje relativně malé množství různorodých záznamů, což ji činí nevhodnou pro sloupcové databáze.

V datech se mohou objevit nové atributy nebo některé atributy mohou chybět, takže data mohou být heterogenní, což je pro dokumentovou databázi přirozené. Dokumentová databáze zároveň podporuje horizontální škálování a shardování dle regionu, což je vhodné pro aplikace pracující s GIS daty.

Databáze byla tedy vybrána vzhledem k možnosti ukládání heterogenní GeoJSON struktury, podpoře indexace geometrie a snadnému přidávání nových dat s možností horizontálního škálování.

Příkaz pro definici

Napojení na mongoDB v jazyce python, vytvoření databáze a kolekce a vytvoření 2dsphere indexu na pole geometry (případně pouze připojení, pokud kolekce existuje):

```
client = MongoClient(mongodb://admin:admin@mongodb:27017/)
db = client[upa_test]
collection = db[hunting]
collection.create_index([("geometry", "2dsphere")])
```

Definici databáze lze vidět ve funkci `create_mongo_connection` v příloženém souboru `mongoDB.py`.

Algoritmus na import dat

Algoritmus na import dat lze vidět ve funkci `load_geojson` v příloženém souboru `mongoDB.py`.

Dotaz nad daty

Dotaz, který najde honitby ve vzdálenosti 5 km od konkrétního bodu:

```
point = {"type": "Point", "coordinates": [13.28946164, 49.76122593]}
pipeline = [
    {
        "$geoNear": {
            "near": point,
            "distanceField": "distance_m",
            "maxDistance": 5000,
            "spherical": True,
            "query": {}
        }
    },
    {
        "$group": {
            "_id": "$properties.NAZEV",
            "distance_m": {"$first": "$distance_m"},
            "hectars": {"$first": "$properties.VYMERHA"},
        }
    },
    {
        "$project": {
            "_id": 0,
            "NAZEV": "$_id",
            "VZDALENOST_KM": {"$round": [{"$divide": ["$distance_m", 1000]}, 3]},
            "VYMERHA_KM": {"$divide": ["$hectars", 100]}
        }
    }
]
```

```

    }
]
results = collection.aggregate(pipeline)

```

Pipeline začíná fází `geoNear`, která vyžaduje `2dsphere` index na geometrii. Tento index byl vytvořen při načítání dat. Dotaz tedy začíná spuštěním `geoNear`, které využívá index pro rychlé filtrování dat podle vzdálenosti. Následuje fáze `group`, která seskupí dokumenty podle názvu honitby, čímž se eliminují duplicity v případě, že má honitba více záznamů. Výsledky jsou poté transformovány pomocí `project` – zde se provádí výpočet vzdálenosti v kilometrech se zaokrouhlením na tři desetinná místa a extrakce názvu a hodnoty `km2`. Klient nakonec obdrží agregační kurzor nad finálním datovým proudem.

Dotaz nad daty lze také vidět ve funkci `query_data` v příloženém souboru `mongoDB.py`.

Grafová databáze - Neo4j

Datová sada a distribuce

- Název: Jízdní řády PMDP
- Url: <https://opendata.plzen.eu/public/opendata/detail/187>
- Distribuce: TXT

Vysvětlení volby

Dataset sestává z několika textových souborů, která obsahují data ve formátu `csv`. Ty obsahují data o jednotlivých linkách městské hromadné dopravy v Plzni, konkrétních jízdách pro jednotlivé linky, zastávkách, posloupností zastávek pro jednotlivé linky a možností přechodu pěšky mezi některými zastávkami.

Jednotlivé zastávky (uzly) a jejich následnost (hrany) pro jednotlivé linky přirozeně tvoří orientovaný graf, ve kterém bude cílem nalézt cestu mezi zastávkami. Klíčové jsou potom dotazy týkající vztahů mezi jednotlivými entitami, např. přes které zastávky se dá z jedné zastávky dostat na jinou či jaké zastávky danou linku tvoří. Sekundární potom budou dotazy týkající se atributů jednotlivých entit, např. které zastávky dané linky jsou přizpůsobeny pro vozíčkáře nebo jaké jsou GPS souřadnice zastávek dané linky.

V kontextu využití dané datové sady bude potřeba zpracovávat velké množství složitějších dotazů, zahrnující prohledání grafové struktury, kterou linky, resp. zastávky tvoří. Data mají spíše statický charakter a frekvence aktualizací bude pravděpodobně v řádech týdnů, či měsíců, při aktualizaci jízdního řádu, vytvoření nových linek a zastávek nebo změně jejich atributů. Konzumenti aplikace využívající databázi obsahující tato data budou s největší pravděpodobností na území města Plzeň. Dalo by se očekávat, že dotazy týkající se zastávek v centru budou častější.

Ze zmíněných NoSQL databází je tedy nejvhodnější grafová databáze, konkrétně Neo4j, která umožňuje data uložit ve formě ohodnoceného orientovaného grafu a efektivně zpracovat dotazy na vztahy mezi jednotlivými uzly. Zároveň databázový produkt Neo4j poskytuje knihovnu implementující požadované grafové algoritmy. Neo4j je optimalizováno pro čtení, včetně složitějších dotazů zahrnující prohledání uložené grafové struktury. Dále Neo4j implementuje cache mechanismus pro časté dotazy. Neo4j community edition nepodporuje sharding, ale to

zde nepředstavuje problém, jelikož datová sada odpovídá grafu představujícímu jeden logický celek. Sharding napříč více uzly by při prohledávání grafu přinesl spíše komplikace. Dalo by se uvažovat o replikaci dat na více uzlů a rozložení zátěže, což je nativně podporováno pouze pro Neo4j enterprise edition.

Příkaz pro definici

Definice úložiště u Neo4j spočívá v definici omezení a následné vytvoření samotných uzlů a hran mezi nimi.

Pro zvolenou datovou sadu byla vybrána následující omezení vynucující unikátnost id jednotlivých uzlů.

```
CREATE CONSTRAINT unique_trip_id IF NOT EXISTS
  FOR (t:Trip) REQUIRE t.trip_id IS UNIQUE;
CREATE CONSTRAINT unique_stop_id IF NOT EXISTS
  FOR (s:Stop) REQUIRE s.stop_id IS UNIQUE;
CREATE CONSTRAINT unique_route_id IF NOT EXISTS
  FOR (r:Route) REQUIRE r.route_id IS UNIQUE;
```

Jednotlivé uzly a hrany jsou potom vytvořeny následovně:

- Route

```
MERGE (r:Route {route_id: '2521'})
SET
  r.agency_id = '1',
  r.route_short_name = '1',
  r.route_long_name = 'Slovany - Bolevec',
  r.route_type = '0',
  r.dir_1_from = 'Slovany',
  r.dir_1_to = 'Bolevec',
  r.dir_2_from = 'Bolevec',
  r.dir_2_to = 'Slovany'
```

- Trip a TripOnRoute

```
MERGE (t:Trip {trip_id: '499851'})
SET t.service_id = '1',
  t.trip_short_name = '1',
  t.direction_id = '0',
  t.wheelchair_accessible = '1'
WITH t
MATCH (r:Route {route_id: '2521'})
MERGE (t)-[:TRIP_ON_ROUTE]->(r)
```

- Stop

```
MERGE (s:Stop { stop_id: '27' })
SET s.stop_code = '54720/1',
  s.stop_name = 'Anglické nábřeží',
```

```
s.stop_lat = '49.74571991',
s.stop_lon = '13.38265038',
s.zone_id = '001',
s.location_type = '0',
s.wheelchair_boarding = '1',
s.original_stop_id = '1'
```

- Transfer

```
MATCH (s1:Stop {stop_id: '25'})
MATCH (s2:Stop {stop_id: '27'})
MERGE (s1)-[:TRANSFER]->(s2)
```

- NextStop

```
MATCH (s1:Stop {stop_id: '746'})
MATCH (s2:Stop {stop_id: '1'})
MERGE (s1)-[:NEXT_STOP]->(s2)
```

- StopOnRoute

```
MATCH (s:Stop {stop_id: '746'})
MATCH (r:Route {route_id: '2521'})
MERGE (s)-[:STOP_ON_ROUTE]->(r)
```

Algoritmus na import dat

Algoritmus pro import dat je implementován v přiloženém souboru **neo4jDB.py**, konkrétně funkce **load_dataset**. Pro linky, zastávky, přesuny mezi zastávkami a jízdy, včetně jejich vztahu k dané lince, lze vytvořit uzel, hranu přímo pro řádky čtené z korespondujících souborů provedením výše uvedených příkazů pro definici hran a uzlů v jazyce Cypher nad vytvořeným sezením. Pro vytvoření hran následnosti zastávek a jejich náležitost do linek je nejdříve z dat v souboru **stop_times.txt** vybrána reprezentativní jízda pro jednotlivé linky a pro tu získána posloupnost zastávek. Tyto posloupnosti jsou potom vztaženy k odpovídajícím linkám a využity pro vytvoření hran následnosti zastávek pro jednotlivé linky.

Dotaz nad daty

Pro Neo4j uvažujeme, že je databáze provozována na jednom uzlu, nebo jsou data pouze replikovány na více uzlech. Na výběru uzlu tedy nezáleží. Po získání požadovaných dat jsou výsledky klientovi zaslány s využitím protokolu Bolt.

Nalezení nejkratší cesty (nejmenší počet přejezdů) mezi 2 zastávkami:

```
MATCH p = ALL SHORTEST
(:Stop {stop_code: '50733/3'})-[:NEXT_STOP*]->(:Stop
                                                    {stop_code: '57135/1'})
RETURN p;
```

Databázový server provede vyhledání uzlů typu Stop s danými stop_code, buď za využití index, pokud byl definován, nebo lineárně. Pokud byly uzly nalezeny, tak s využitím zabudované knihovny dojde k nalezení nejkratších cest mezi danými uzly výhradně s využitím hran typu NEXT_STOP. Výsledkem dotazu je/jsou nejkratší cesta/cesty jako posloupnost zastávek a hran typu NEXT_STOP, kdy klientovi je prezentována nejkratší posloupnost zastávek, skrze které se dostat na cílovou zastávku.

Výpis GPS souřadnic jednotlivých zastávek linky Slovany - Bolevec v pořadí daným směrem jízdy.

```
MATCH (s:Stop)-[e:STOP_ON_ROUTE]->(r:Route
                                                    {route_long_name: 'Slovany - Bolevec'})
ORDER BY e.order
RETURN s.stop_lat, s.stop_lon;
```

Databázový server provede vyhledání uzlů typu Route s danými route_long_name, buď za využití indexu, pokud byl definován, nebo lineárně. Následně server provede prohledání grafové struktury od uzlu reprezentujícího danou linku za využití hran typu STOP_IN_ROUTE, kdy tato relace zahrnuje atribut order určující pořadí zastávek v rámci dané linky. Nalezené uzly reprezentující zastávky jsou seřazeny podle atributu order náležícího hraně, která k němu vede. Z jednotlivých uzlů zastávek jsou získány GPS souřadnice a odeslány klientovi pro možné zobrazení na mapě.

Časová databáze - InfluxDB

Datová sada a distribuce

- Název: Detektory zvuku - Nerudova
- Url: <https://opendata.plzen.eu/public/opendata/detail/209>
- Distribuce: CSV

Vysvětlení volby

Jedná se o data z měřiče detektoru zvuku ve formátu CSV. Každý záznam obsahuje čas pořízení, identifikátor zařízení, jeho název a hodnoty zvuku (okolní energie, maximální a minimální energie). Nové záznamy se ukládají každé 3 minuty a datová sada obsahuje tisíce záznamů. Každý záznam má čas jeho pořízení, což je ideální pro ukládání v časové databázi.

V datech se přibývají nové záznamy pravidelně a existující záznamy se nemění. Časová databáze podporuje downsampling a agregaci, což je výhodné pro analýzu změn dat v čase.

Databáze byla vybrána, neboť data tvoří časovou řadu a mají pevnou strukturu. InfluxDB nad těmito daty umožňuje rychlé dotazy na časové intervaly (na rozdíl od sloupcové DB). Zároveň podporuje horizontální škálování a shardování podle času či měřícího zařízení, což je vhodné pro velký objem naměřených dat stejného charakteru.

Databáze byla tedy vybrána vzhledem k možnosti ukládání časově řazených měření, rychlé agregaci a filtraci dat a snadnému horizontálnímu škálování.

Příkaz pro definici

Příkaz pro definici databáze:

```
influx setup \  
  --username admin \  
  --password 12341234 \  
  --token my-token \  
  --org localInfluxTest \  
  --bucket SoundDetectorTest
```

Příkaz pro vytvoření bucketu:

```
influx bucket create --name SoundDetectorTest
```

Setup databáze lze také vidět v příloženém souboru **docker-compose.yml**. Napojení na databázi a tvorbu bucketu lze také vidět ve funkci **create_influx_client** a **create_bucket** v příloženém souboru **influxDB.py**. Definice struktury tabulky je provedena automaticky na základě nahraných dat.

Algoritmus na import dat

Algoritmus na import dat lze vidět ve funkci **load_csv** v příloženém souboru **influxDB.py**.

Dotaz nad daty

Dotaz, který získá průměrnou hodnotu energie zvuku za každý den v září roku 2025 podle zařízení:

```
from(bucket: "bucketName")  
|> range(start: 2025-08-31, stop: 2025-09-30)  
|> filter(fn: (r) => r._measurement == "sound_measurement")  
|> filter(fn: (r) => r._field == "energy")  
|> aggregateWindow(every: 1d, fn: mean, createEmpty: false)  
|> group(columns: ["device_name"])  
|> keep(columns: ["_time", "_value", "device_name"])
```

Engine InfluxDB analyzuje dotaz a určí, který bucket a jaký časový rozsah jsou potřeba. Následně na základě těchto údajů zjistí, které uzly obsahují požadovaná data (rozdělení na uzly probíhá podle zvoleného tagu – v tomto případě podle `serial_number` a `device_number`). Příslušné uzly poté načtou lokálně data ze svých shardů, aplikují filtry a provedou agregace a seskupení. Částečné výsledky se následně sloučí. Řídící uzel pak serializuje finální tabulku obsahující pouze požadované sloupce a doručí ji klientovi prostřednictvím rozhraní HTTP(S). Aplikace obdrží data deserializovaná do tabulek a iteruje přes jednotlivé záznamy.

Dotaz nad daty lze také vidět ve funkci **query_data.py** v příloženém souboru **influxDB.py**.